

Lab 2 - ELA408

Kim Svedberg



I. EXTENDED KALMAN FILTER

Extended Kalman Filter (EKF) handles non-linear system dynamics and measurement models by linearizing about the current mean and covariance using Jacobians. The parameters used for the EKF can be seen in Table I.

TABLE I
NUMERICAL EKF PARAMETERS USED.

Parameters	Values
Wheel base (l)	0.1m
Distance noise (σ_d)	0.5m
Heading noise (σ_θ)	$10^\circ \frac{\pi}{180^\circ} \text{rad}$
Range measurement noise (σ_r)	0.5m
Bearing measurement noise (σ_β)	$10^\circ \frac{\pi}{180^\circ} \text{rad}$

The EKF process can be divided into two steps:

- **Prediction:** Uses the motion model and the previous state estimate to propagate the state and covariance forward in time.
- **Correction:** Uses the latest sensor measurements to landmarks to update the estimate, reducing uncertainty by weighting the prediction against the measurements.

A. Prediction

The robot state is defined by its pose $\mathbf{x} = [x, y, \theta]^\top$. The differential drive motion model propagates the state as:

$$x(k+1) = x(k) + \left(\frac{\Delta s_r + \Delta s_l}{2} + v_d \right) \cdot \cos(\theta(k)) \quad (1)$$

$$y(k+1) = y(k) + \left(\frac{\Delta s_r + \Delta s_l}{2} + v_d \right) \cdot \sin(\theta(k)) \quad (2)$$

$$\theta(k+1) = \theta(k) + \frac{\Delta s_r - \Delta s_l}{l} + v_\theta \quad (3)$$

where process noises are $v_d \sim \mathcal{N}(0, \sigma_d^2)$ and $v_\theta \sim \mathcal{N}(0, \sigma_\theta^2)$, Δs_r and Δs_l is the measured odometric data coming from the wheel encoders, and k is the time step. The angle θ is always wrapped between $[-\pi, \pi]$ using atan2 . The predicted state and covariance are:

$$\hat{\mathbf{x}}(k+1|k) = \mathbf{f}(\hat{\mathbf{x}}(k|k), \mathbf{u}(k)) \quad (4)$$

$$\hat{\mathbf{P}}(k+1|k) = \mathbf{F}_x \hat{\mathbf{P}}(k|k) \mathbf{F}_x^\top + \mathbf{F}_v \hat{\mathbf{V}} \mathbf{F}_v^\top \quad (5)$$

where $\hat{\mathbf{x}}(k+1|k)$ is the propagated state, $\hat{\mathbf{P}}(0|0) = \text{diag}(10, 10, \pi^2)$ is the initial covariance, the process noise

covariance is $\hat{\mathbf{V}} = \mathbf{I}_2$, and the Jacobians of $\mathbf{f}$ with respect to the state and noise are:

$$\mathbf{F}_x = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{v}=\mathbf{0}} = \begin{bmatrix} 1 & 0 & -\left(\frac{\Delta s_r + \Delta s_l}{2}\right) \sin(\theta(k)) \\ 0 & 1 & \left(\frac{\Delta s_r + \Delta s_l}{2}\right) \cos(\theta(k)) \\ 0 & 0 & 1 \end{bmatrix} \quad (6)$$

and

$$\mathbf{F}_v = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{v}} \right|_{\mathbf{v}=\mathbf{0}} = \begin{bmatrix} \cos(\theta(k)) & 0 \\ \sin(\theta(k)) & 0 \\ 0 & 1 \end{bmatrix} \quad (7)$$

B. Correction

Given $L = 6$ landmarks, each producing a range and bearing measurement, the full measurement vector $\mathbf{z}(k+1) \in \mathbb{R}^{12}$ stacks $[r_i, \beta_i]^\top$ for $i = 1, \dots, 6$. The predicted measurement for landmark i located at $\mathbf{p}_i = [x_i, y_i]^\top$ is:

$$\mathbf{h}(\mathbf{x}, \mathbf{p}_i) = \begin{bmatrix} r_i \\ \beta_i \end{bmatrix} = \begin{bmatrix} \sqrt{(x_i - x)^2 + (y_i - y)^2} \\ \text{atan2}\left(\frac{y_i - y}{x_i - x}\right) - \theta \end{bmatrix} + \begin{bmatrix} w_r \\ w_b \end{bmatrix} \quad (8)$$

with measurement noise $[w_r, w_b]^\top \sim \mathcal{N}(\mathbf{0}, \mathbf{W})$, where $\mathbf{W} = \text{diag}(\sigma_r^2, \sigma_\beta^2)$.

The innovation equation compares the sensor measurement with the predicted measurement and is given by:

$$\mathbf{v} = \mathbf{z}(k+1) - \mathbf{h}(\hat{\mathbf{x}}(k+1|k), \mathbf{p}_i) \quad (9)$$

The Jacobian of $\mathbf{h}$ with respect to the state, evaluated for landmark i , is:

$$\mathbf{H}_{x,i} = \left. \frac{\partial \mathbf{h}_i}{\partial \mathbf{x}} \right|_{\mathbf{w}=\mathbf{0}} = \begin{bmatrix} -\frac{x_i - x}{r_i} & -\frac{y_i - y}{r_i} & 0 \\ \frac{y_i - y}{r_i^2} & -\frac{x_i - x}{r_i^2} & -1 \end{bmatrix} \quad (10)$$

where $r_i = \sqrt{(x_i - x)^2 + (y_i - y)^2}$. The full $\mathbf{H}_x \in \mathbb{R}^{12 \times 3}$ is obtained by stacking $\mathbf{H}_{x,i}$ for all six landmarks. Similarly, $\mathbf{H}_w = \mathbf{I}_{12 \times 12}$ is the identity matrix that maps each $[w_r, w_\beta]^\top$ into the full measurement space.

The innovation covariance and Kalman gain are:

$$\mathbf{S} = \mathbf{H}_x \hat{\mathbf{P}}(k+1|k) \mathbf{H}_x^\top + \mathbf{H}_w \hat{\mathbf{W}} \mathbf{H}_w^\top \quad (11)$$

$$\mathbf{K} = \hat{\mathbf{P}}(k+1|k) \mathbf{H}_x^\top \mathbf{S}^{-1} \quad (12)$$

The Kalman gain $\mathbf{K}$ decides how much to trust the correction. If the sensors are noisy, $\mathbf{K}$ is small and we barely move from the prediction, while if the model is noisy, $\mathbf{K}$ is large

and we lean heavily on the measurement. Finally, the state and covariance are updated/correction by:

$$\hat{\mathbf{x}}(k+1|k+1) = \hat{\mathbf{x}}(k+1|k) + \mathbf{K}\mathbf{v} \quad (13)$$

$$\hat{\mathbf{P}}(k+1|k+1) = \hat{\mathbf{P}}(k+1|k) - \mathbf{K}\mathbf{H}_x\hat{\mathbf{P}}(k+1|k) \quad (14)$$

C. EKF Results

The EKF trajectory can be seen in Figure 1. The estimation closely follows the trajectory and reaches the goal.

Figure 2 shows the x , y , and θ tracking. The robot follows the x and y perfectly, but struggles to keep its heading θ totally stable.

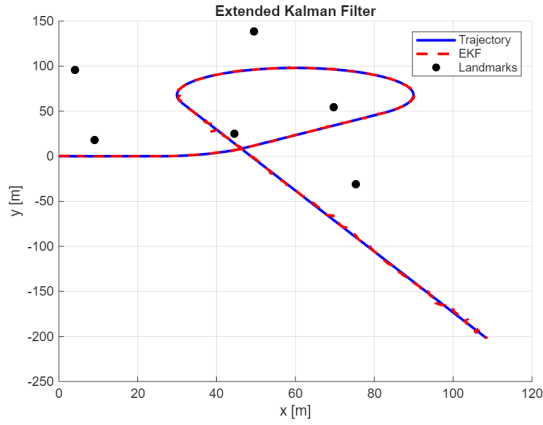


Fig. 1. Extended Kalman Filter trajectory (blue line), estimated position (red line), and landmarks (black dots).

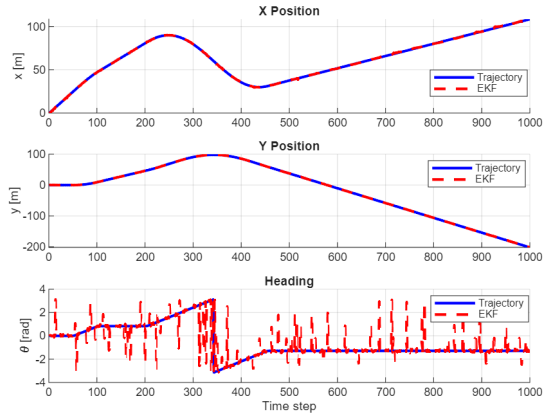


Fig. 2. Extended Kalman Filter plot of x , y , and θ . All reaches follow the trajectory.

II. PARTICLE FILTER

The Particle Filter (PF) can be used if the initial position is not known and the state follows a non-Gaussian distribution.

The PF process can be divided into five steps:

- 1) **Initialization:** Initialize the particle set across the map.
- 2) **Prediction:** Predict the movement of particles using a motion model.
- 3) **Update:** Update the weights based on sensor measurements.

4) **Normalization:** Normalize the weights and compute the estimated position.

5) **Resampling:** Resample the best particles.

Step 2 through 5 are repeated until the robot reaches its goal. The parameters used for the PF can be seen in Table II.

TABLE II
NUMERICAL PF PARAMETERS USED.

Parameters	Values
Wheel base (l)	0.1m
Uniform distribution (a_d)	-1m
Uniform distribution (b_d)	3m
logistic distribution (μ_θ)	$30^\circ \frac{\pi}{180^\circ} \text{rad}$
logistic distribution (s_θ)	$45^\circ \frac{\pi}{180^\circ} \text{rad}$
Range measurement noise (s_d)	1m
Bearing measurement noise (s_β)	$20^\circ \frac{\pi}{180^\circ} \text{rad}$

A. Initialize Particles

PF starts by randomly distributing N particles across the whole map, where N is the number of particles. Each particles weight is initially the same $w_i = \frac{1}{N}$, meaning that the robot can be anywhere.

B. Predict Movement

The particles are propagated according to the motion model:

$$\hat{\mathbf{x}}(k+1) = \mathbf{x}(k) + \begin{bmatrix} (\delta_d + r_{d_i}) \cdot \cos \theta(k) \\ (\delta_d + r_{d_i}) \cdot \sin \theta(k) \\ \delta_\theta + r_{\theta_i} \end{bmatrix} \quad (15)$$

where r_d and r_θ represent the process noise for distance and heading.

C. Update Weights

The measurement model $\mathbf{h}(\mathbf{x}_i, \mathbf{p}_j)$ compares the particle's predicted landmark range and bearing to the actual sensor measurements $\mathbf{z}$:

$$\mathbf{z} = \mathbf{h}(\mathbf{x}_i, \mathbf{p}_j) = \begin{bmatrix} \sqrt{(y_i - y)^2 + (x_i - x)^2} \\ \text{atan2}\left(\frac{y_i - y}{x_i - x}\right) - \theta \end{bmatrix} + \begin{bmatrix} w_r \\ w_\beta \end{bmatrix} \quad (16)$$

The weights are then updated by multiplying the probabilities for each of the L landmarks:

$$w_i = \prod_j \frac{1}{\sigma \sqrt{2\pi}} \exp \left(-\frac{1}{2} \left(\frac{h(\mathbf{x}_i, p_j) - \mathbf{z}_j}{\sigma} \right)^2 \right) \quad (17)$$

where w_i is the weight of particle i (larger weight means a higher chance that the robot is there), L is the number of landmarks, σ is the standard deviation of the measurement noise, $h(\mathbf{x}_i, p_j)$ is the estimated measurement from particle $\mathbf{x}_i$ to the known position of landmark p_j , and $\mathbf{z}_j$ is the sensor measurement from the robots position to the landmark j .

D. Normalize and Estimate

To prevent weight divergence, the weights need to be normalized before estimating the position:

$$w_i = \frac{w_i}{\sum_{i=1}^N w_i} \quad (18)$$

The estimated position $\hat{x}$ and $\hat{y}$ are calculated as the weighted mean of the particle set, while the estimated heading $\hat{\theta}$ is calculated as the weighted mean of the particle angles on a unit circle (using atan2 to wrap the angles between $[-\pi, \pi]$):

$$\hat{x} = \sum_{i=1}^N x_i w_i \quad (19)$$

$$\hat{y} = \sum_{i=1}^N y_i w_i \quad (20)$$

$$\hat{\theta} = \text{atan2} \left(\sum_{i=1}^N w_i \sin \theta_i, \sum_{i=1}^N w_i \cos \theta_i \right) \quad (21)$$

E. Resample

The particle set is then resampled because otherwise we will have one particle with a weight close to 1, and all other particles would have a weight close to 0. The low variance resampling algorithm was implemented, which selects a new population by picking a single random starting point and placing evenly spaced pointers across the weighted distribution of particles. This ensures that higher weight particles are guaranteed a spot while reducing the noise:

Algorithm 1 Low variance re-sampling ($\mathbf{x}, \mathbf{w}$)

```

 $\mathbf{x}_{k+1} = \emptyset$ 
 $r = \text{rand}(0; N^{-1}), c = w_1, j = 1$ 
for  $i = 1$  to  $N$  do
     $U = r + (i - 1) \cdot N^{-1}$ 
    while  $U > c$  do
         $j = j + 1$ 
         $c = c + w_j$ 
    add  $x_j$  to  $\mathbf{x}_{k+1}$ 
Return  $\mathbf{x}_{k+1}$ 

```

F. PF Results

The particle filter trajectories can be seen in Figure 3. All PF reached the goal, but 100 particle filter had less stable heading.

Figure 4 shows the x, y , and θ tracking. While all filters eventually follow the desired trajectory (blue line), the 100-particle filter is slightly slower to converge and has higher noise in the heading (θ).

The computational time was measured for the filters. As shown in Table II-F, the 100-particle filter was the fastest (0.307s), while the 3000-particle filter required the most time (0.976s). Times may vary depending on the computer used.

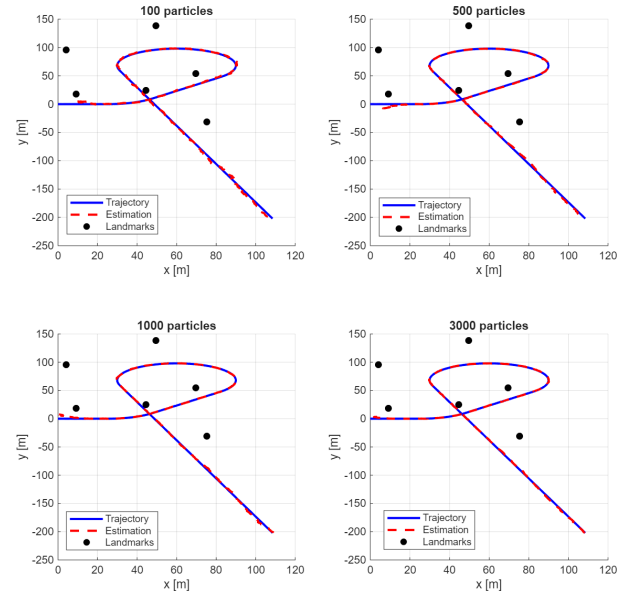


Fig. 3. Particle filter trajectories (blue line), estimated position (red line), and landmarks (black dots). Four different filters were tested: 100, 500, 1000, 3000 particles. All reaches the goal.

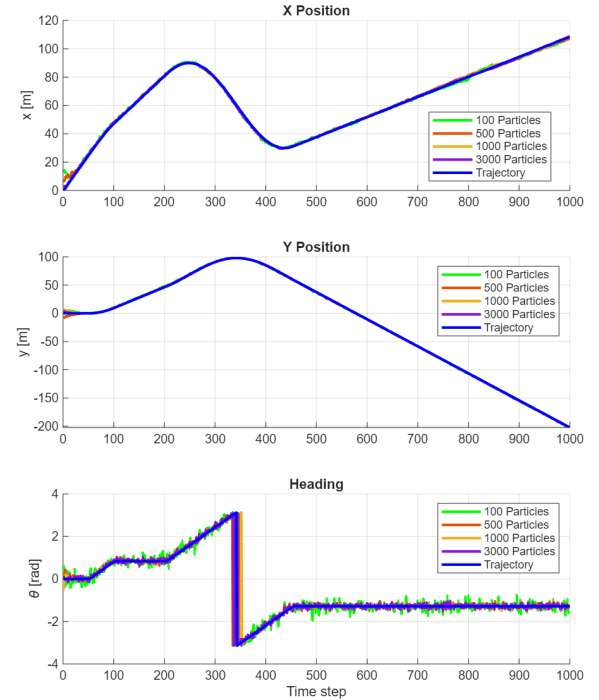


Fig. 4. Particle filter plot of x, y , and θ . Four different filters: 100, 500, 1000, 3000 particles. All follow the trajectory.

TABLE III
ELAPSED TIME FOR DIFFERENT PARTICLE COUNTS.

Particle Count	Elapsed Time (s)
100 Particles	0.307846
500 Particles	0.501095
1000 Particles	0.618775
3000 Particles	0.976924